

Employee Management System

Exercise 1: Project Setup

Dependencies:

- Spring Web
- Spring Data JPA
- H2 Database
- Lombok

application.properties

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=update
```

Exercise 2: Entity Classes

Department.java

```
@Entity
@Table(name="department")
public class Department {
    @Id @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy="department")
    private List<Employee> employees;
}
```

Employee.java

```
@Entity
@Table(name="employee")
```

```

public class Employee{
    @Id @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;

    @ManyToOne
    @JoinColumn(name="department_id")
    private Department department;
}

```

Explanation:

@Entity marks a class as a JPA entity. @Table maps it to a database table. @Id identifies the primary key. @GeneratedValue enables automatic ID generation. @ManyToOne and @OneToMany establish the relationship between Employee and Department.

Exercise 3: Repository

EmployeeRepository.java

```

public interface EmployeeRepository extends JpaRepository<Employee,Long>{
    List<Employee> findByName(String name);
    List<Employee> findByDepartmentId(Long id);
}

```

DepartmentRepository.java

```

public interface DepartmentRepository extends JpaRepository<Department,Long>{
    Department findByName(String name);
}

```

JpaRepository provides CRUD methods such as save(), findAll(), findById(), deleteById() and count(). Derived query methods are created automatically from method names.

Exercise 4: CRUD Operations

Service methods:

saveEmployee(), getAllEmployees(), getEmployeeById(), updateEmployee(), deleteEmployee().

Controller endpoints:

POST /employees

GET /employees

GET /employees/{id}

PUT /employees/{id}

DELETE /employees/{id}

Example:

@PostMapping

```
public Employee saveEmployee(@RequestBody Employee employee){  
    return employeeService.saveEmployee(employee);  
}
```

@DeleteMapping("/{id}")

```
public void deleteEmployee(@PathVariable Long id){  
    employeeService.deleteEmployee(id);  
}
```

Exercise 5: Query Methods

Derived Query Methods

findByName(String name)

findByEmail(String email)

findByDepartmentId(Long id)

Using @Query

```
@Query("SELECT e FROM Employee e WHERE e.department.id=:id")  
List<Employee> findEmployeesByDepartment(@Param("id") Long id);
```

Named Query

```
@NamedQuery(  
    name="Employee.findByEmail",  
    query="SELECT e FROM Employee e WHERE e.email=:email"  
)
```

Exercise 6: Pagination and Sorting

Pagination:

```
Pageable pageable = PageRequest.of(0,5);
```

```
Page<Employee> employees = employeeRepository.findAll(pageable);
```

Sorting:

```
Sort sort = Sort.by("name").ascending();
```

```
List<Employee> employees = employeeRepository.findAll(sort);
```

Pagination with Sorting:

```
Pageable pageable = PageRequest.of(0,5,Sort.by("name").descending());
```

```
Page<Employee> page = employeeRepository.findAll(pageable);
```

Advantages:

- Faster retrieval of records
- Reduces memory usage
- Improves user experience
- Efficient for large datasets

Exercise 7: Enabling Entity Auditing

Entity auditing automatically records when an entity is created or modified and by whom.

Steps:

1. Enable auditing using `@EnableJpaAuditing` in the main application class.
2. Add auditing fields in the entity.

Example:

```
@CreateDate
```

```
private LocalDateTime createdAt;
```

```
@LastModifiedDate
```

```
private LocalDateTime lastModifiedDate;
```

```
@CreatedBy
```

```
private String createdBy;
```

```
@LastModifiedBy
```

```
private String lastModifiedBy;
```

Advantages:

- Tracks record history
- Improves security and accountability

- Useful for enterprise applications

Exercise 8: Creating Projections

Projections are used to retrieve only the required fields instead of the complete entity.

Interface-based Projection

```
public interface EmployeeView {  
    String getName();  
    String getEmail();  
}
```

Repository Method

```
List<EmployeeView> findByDepartmentId(Long id);
```

Class-based Projection

```
public class EmployeeDTO {  
    private String name;  
    private String email;  
  
    public EmployeeDTO(String name, String email) {  
        this.name = name;  
        this.email = email;  
    }  
}
```

Advantages:

- Improves performance
- Reduces unnecessary data transfer
- Faster query execution

Exercise 9: Customizing Data Source Configuration

Spring Boot automatically configures the datasource using application.properties.

Example:

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.username=sa
spring.datasource.password=password
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
```

Multiple Data Sources:

- Configure separate DataSource beans.
- Use @Primary for the default datasource.
- Configure separate EntityManagerFactory and TransactionManager for each datasource.

Benefits:

- Connect to multiple databases.
- Better scalability.
- Supports distributed applications.

Exercise 10: Hibernate-Specific Features

Hibernate provides additional features beyond standard JPA.

1. Hibernate Annotations

@DynamicUpdate

@CreationTimestamp

@UpdateTimestamp

Example:

@CreationTimestamp

private LocalDateTime createdOn;

@UpdateTimestamp

private LocalDateTime updatedOn;

2. Hibernate Dialect

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
```

3. Batch Processing

```
spring.jpa.properties.hibernate.jdbc.batch_size=20
```

Benefits:

- Faster bulk insert/update operations.
- Reduces database round trips.
- Improves application performance.